

Osservazioni sui Benchmark degli Algoritmi di Ordinamento

Analizzando i tempi di esecuzione sui vettori generati, per dimensioni che variano da $N = 4$ a $N = 90$, emergono evidenze pratiche che illustrano il comportamento reale degli algoritmi al netto della loro pura complessità asintotica. L'analisi confronta la compilazione con ottimizzazioni (Grafico 1 - Modalità Release) e senza ottimizzazioni (Grafico 2 - Modalità Debug).

- **Impatto delle ottimizzazioni (Release vs Debug):** Confrontando i due grafici si nota un drastico abbattimento dei tempi assoluti nella modalità Release (passando da un ordine di grandezza di 10^{-5} a 10^{-6} secondi). L'andamento delle curve rimane coerente, dimostrando che le ottimizzazioni del compilatore riducono il tempo della singola istruzione ma non alterano la classe di complessità dell'algoritmo.
- **L'esplosione del Bubble Sort e l'efficienza del Selection Sort:** Come evidente in entrambi i grafici, il Bubble Sort subisce una rapida impennata tipica degli algoritmi $O(N^2)$. Al contrario, il Selection Sort, pur essendo anch'esso $O(N^2)$, mostra una crescita molto più contenuta e lineare in questo range. Ciò è dovuto al fatto che il Selection Sort effettua solo $O(N)$ scambi in memoria contro gli $O(N^2)$ del Bubble Sort; avendo un costo di scrittura ridotto, la sua inefficienza quadratica non è ancora pienamente visibile a $N = 90$.
- **Il paradosso del Mergesort per piccoli N :** Un dato molto interessante visibile nei grafici è che il Mergesort (teoricamente $O(N \log N)$) risulta più lento del Selection Sort e dell'Insertion Sort per quasi tutto il range analizzato. Questo comportamento è causato dal massiccio *overhead* costante dovuto alle continue allocazioni e deallocazioni in memoria dei vettori ausiliari ad ogni chiamata ricorsiva, un costo che non viene ammortizzato per vettori così piccoli.
- **Prestazioni eccellenti dell'Insertion Sort e del Quick 2.0:** Per tutto il range $N \leq 90$, l'Insertion Sort registra tempi estremamente bassi, risultando l'algoritmo in assoluto più efficiente tra quelli base, complice la sua ottima località spaziale (cache). Basandosi su questa efficienza per piccoli segmenti, il Quicksort Ibrido (Quick 2.0) delega i casi base proprio all'Insertion Sort. Le curve finali dimostrano il successo di questa sinergia: il Quick 2.0 si posiziona stabilmente sul fondo del grafico, performando meglio del Quick Sort puro e risultando del tutto competitivo con la funzione `std::sort` della libreria standard.

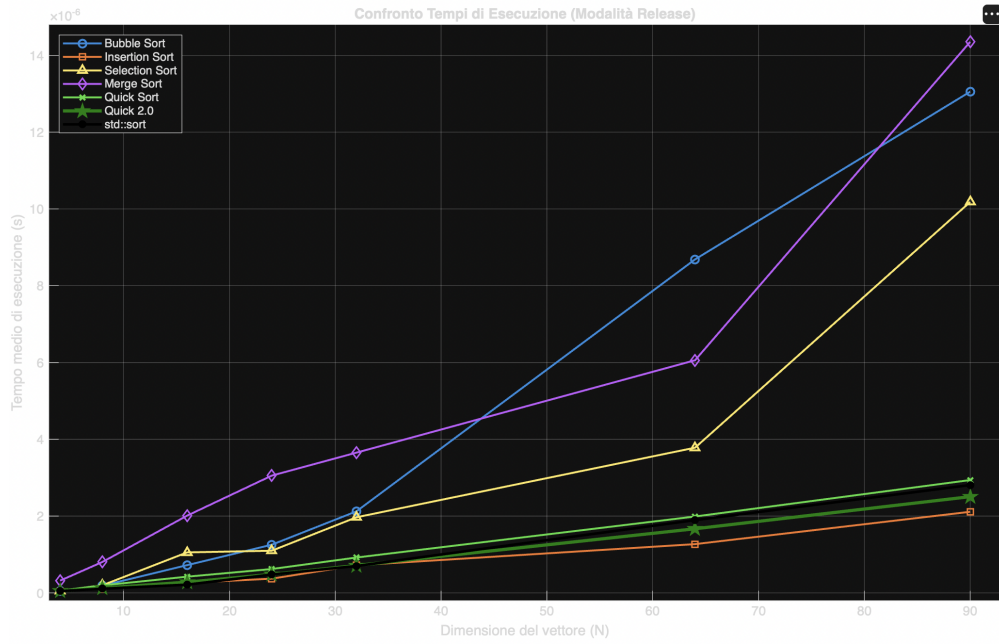


Figure 1: Confronto dei tempi di esecuzione in modalità **Release** (con ottimizzazioni).

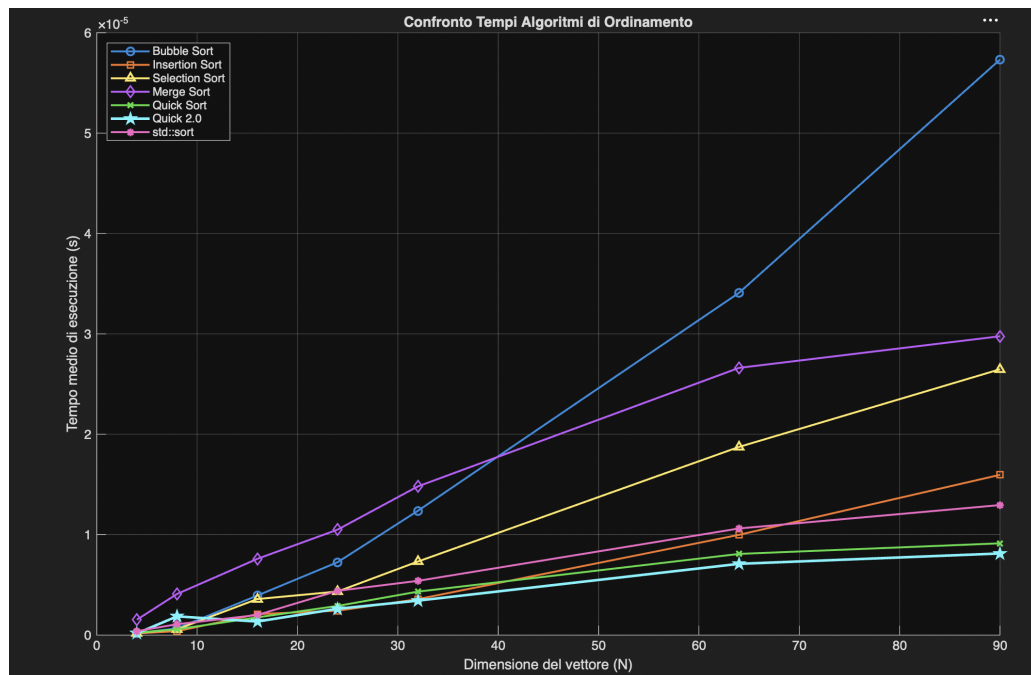


Figure 2: Confronto dei tempi di esecuzione in modalità **Debug** (senza ottimizzazioni).